Java OOP Laboratory Exercise

Campus Café Ordering System

Name: Nadal Cyphen R

Section: _____

Date: November 9, 2026

Topics: Conditional Statements, Classes and Objects, Constructors, Encapsulation, Polymorphism, and Abstraction

1. Concept Explanation

Concept	Where and why it is used
if Statement	Used in **Pastry.computePrice()** to add the 10-peso warming fee and in **Meal.computePrice()** to add the 20-peso upsize fee. A single `if` is enough because nothing changes when the condition is false.
if-else Statement	Used in **MenuItem.setBasePrice()** to reject negative prices, **Beverage.computePrice()** to choose the S/M/L price, **Order.addItem()** to find a free slot, and **Order.computeTotal()** to apply the student discount.
Classes and Objects	The program has 7 classes, each in its own file. **CafeApp.main()** creates the objects: 2 customers, 6 menu items and 2 orders. Classes group data and behavior so each object models one real thing in the cafe.
Attributes and Behaviors	Every class has instance variables (for example **Customer.loyaltyPoints**) and instance methods (for example **addPoints()**), so each object stores its own state and acts on it.
Object Interaction	**Order** stores a **Customer** and up to three **MenuItem** objects. **printReceipt()** calls **computePrice()** on each item and **addPoints()** on the customer, so the objects cooperate instead of working alone.
Java Naming Conventions	PascalCase for classes (**MenuItem**), camelCase for methods and variables (**computeSubtotal**), and UPPER_SNAKE_CASE for constants (**WARMING_FEE**, **MAX_ITEMS**), which keeps the code readable and consistent.
Default Constructor	**MenuItem()**, **Beverage()**, **Pastry()**, **Meal()** and **Customer()** set safe starting values. **CafeApp** uses them for the Hot Chocolate, Cookie and Beef Tapa, which are then completed with setters.
Parameterized Constructor	All MenuItem subclasses, **Customer** and **Order** accept values so an object is complete in one step; subclasses pass shared data up with `super(...)`.
Constructor Overloading	**Customer** has three constructors: **Customer()**, **Customer(String)** and **Customer(String, boolean)**, so a customer can be created with as much or as little detail as is known.
public / private / protected	All fields are **private**, except **name** and **basePrice** in MenuItem, which are **protected** so subclasses can use them in pricing. Methods are **public** so other classes can use the object.
Getters, Setters, Data Hiding	No field is public. Getters read the data and setters validate it first (negative prices, bad sizes, negative sides and negative points are rejected), so an object can never hold invalid data.
Method Overloading	**Customer.addPoints(int)** adds points directly while **addPoints(double)** converts pesos to points; **Order.computeTotal()** and **computeTotal(double)** differ by the voucher. Same name, different parameters.
Method Overriding	**Beverage**, **Pastry** and **Meal** each override **computePrice()** and **getDescription()** with `@Override`, so every item type prices and describes itself correctly.
Abstraction	**MenuItem** is `abstract` with the abstract method **computePrice()**. A generic item is never ordered, and every subclass is forced to define its own pricing.

2. Program Output

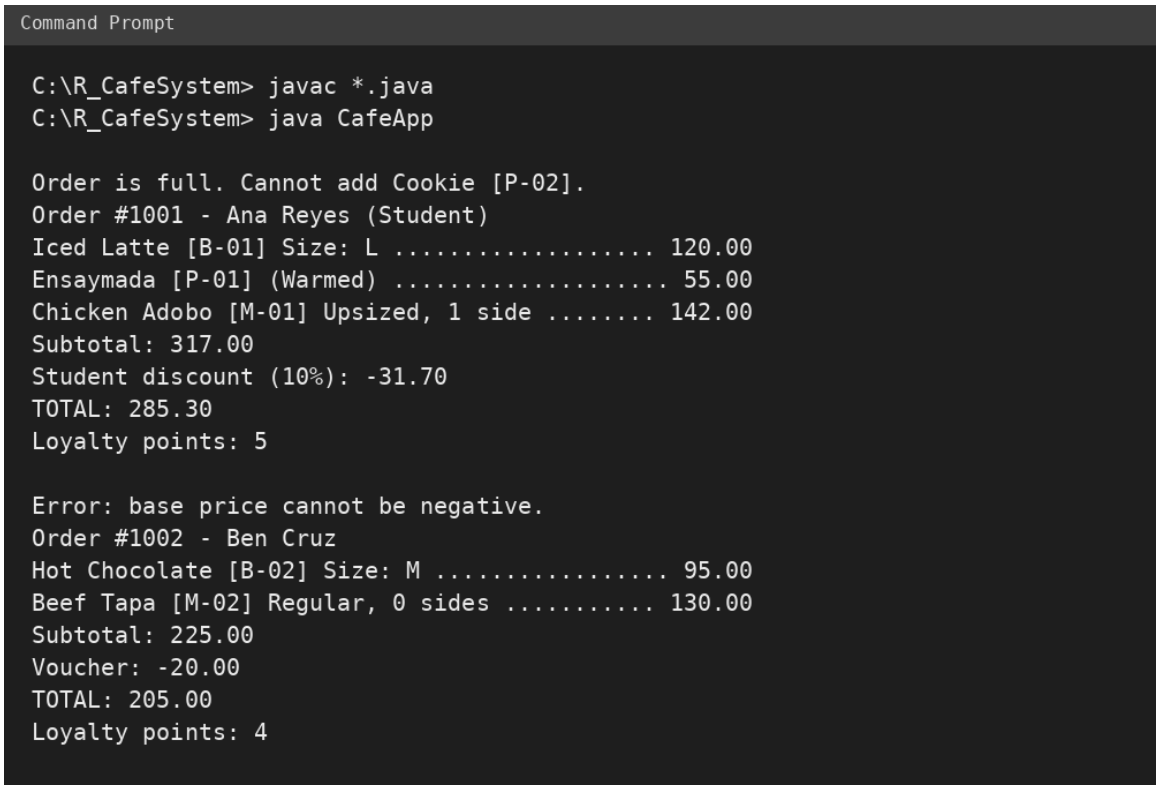


Figure 1. Output of `java CafeApp`: Order #1001 totals 285.30 (5 points) and Order #1002 totals 205.00 (4 points).

3. Challenges Encountered and How I Solved Them

1. Showing a voucher on the receipt. printReceipt() takes no parameters, yet Order #1002 must show a voucher. I stored the voucher in a private **voucher** field of Order with a validated **setVoucher()**, and the receipt calls **computeTotal(double)** only when a voucher exists.

2. Points awarded twice. Because **printReceipt()** adds the loyalty points, printing the same receipt again would add them again. I added a private **pointsAwarded** flag so points are given once per order.

3. Rounding errors in money. Computing 10% off and then dividing by 50 can leave tiny floating-point errors that might remove a point at an exact multiple of 50. I added a small tolerance in **addPoints(double)** and used `%.2f` to print amounts.

4. Invalid data in constructors. A bad value could bypass validation if a constructor assigned fields directly. The parameterized constructors now pass values through the validated setters, so a negative price or illegal size is rejected there too.

5. Compiling anywhere. The program must run with plain `javac` and `java CafeApp`. I used no `java.util` imports or collections, only ASCII characters, and I checked that the code compiles cleanly, including a Javadoc check.